TickHunter vs GodZillaKilla

Order Management, ATM Strategy & ATR Trailing Stops — Side-by-Side Review

---

1. ATM Strategy

Fundamental API Difference

The two codebases use different NT8 ATM APIs with very different ownership models.

	GodZillaKilla	TickHunter
NT8 API	Strategy-class AtmStrategyCreate()	Indicator-class StartAtmStrategy()
ID tracking	atmStrategyId + orderId strings	atmStrategyName (template name only)
Confirmation	Callback lambda on create	Fire-and-forget — no callback
Position monitor	GetAtmStrategyMarketPosition()	RealPositionService.PositionCount
Close ATM	AtmStrategyClose(atmStrategyId)	NT8 owns it — not closed in code
Template source	AtmStrategy property (param)	Loaded live from ChartTrader UI

Lifecycle Protection

- **GodZillaKilla** Defense #3 — if the AtmStrategyCreate callback never fires (silently rejected), the stale atmStrategyId is cleared after 10 seconds to prevent "ID does not exist" error floods that can saturate the log mutex and starve the UI thread.
- **GodZillaKilla** Defense #8 — mid-trade staleness detection on every tick. If the ATM reports Flat but the account still holds a position (HDS bounce), WriteTradeLogRecord is called and the position is force-flattened at account level.
- **TickHunter** has no equivalent stale-ID protection — because it holds no ID. Once StartAtmStrategy() fires, NT8 and ChartTrader own the lifecycle entirely.

Martingale Recovery

- **GodZillaKilla** tracks a second martingaleAtmStrategyId. After a stop-loss, a recovery ATM is created in the opposite direction using a separate MartingaleAtmStrategy template.
- **TickHunter** has no martingale concept.

---

2. Order Entry

	GodZillaKilla	TickHunter
Entry order type	Market only via AtmStrategyCreate()	Market, StopMarket, or Limit via account.CreateOrder() + Submit()
Entry timing	Pending signal pattern: Queued bar 0 close, fired bar 1 tick series	Immediate submission on trigger

	GodZillaKilla	TickHunter
Multi-order queuing	Not supported (one active ATM at a time)	Pop/Drop delayed entry: Up to 5 pending stop orders queued & staged
TP/SL creation	Delegated entirely to ATM template	Scheduled with configurable delay (TakeProfitAddOnDelaySeconds) to batch partial fills

TickHunter's **delayed TP/SL batching** is a noteworthy pattern: if multiple partial fills arrive within the delay window, the timer resets so a single TP/SL order is created for the full accumulated position rather than one per fill. GodZillaKilla sidesteps this entirely by delegating bracket management to the ATM template.

---

3. How TickHunter Uses ATR to Change the Stop

This is **not** a traditional trailing stop. TickHunter implements a **volatility-adaptive stop anchored to entry price** that recalculates every bar as ATR changes, plus a separate **ATR-based breakeven trigger** that fires once price moves far enough in favor.

Step 1 — ATR Value Update (per bar)

```
// Line 3190 atrValue = atrBuffer[1]; // previous bar's ATR, updated once per bar
```

One bar delayed. Every stop calculation reads this single value.

Step 2 — Initial Stop Placed at Entry

```
// Lines 18714-18722 int newATRStopLossTicks = CalculateATRTicks(atrValue,
StopLossInitialATRMultiplier, ticksPerPoint); stopLossTicks = Math.Max(newATRStopLossTicks,
StopLossInitialTicks); // floor at fixed minimum newStopLossPrice =
CalculateStopLossPrice(marketPosition, averagePrice, stopLossTicks, ...);
```

```
// CalculateATRTicks — Lines 18441-18451 atrTicks = (int)((atrValue * ticksPerPoint) *
atrMultiplier);
```

Stop is placed at **entry price +/- (ATR x multiplier)**, with a fixed-tick floor so it can never be tighter than StopLossInitialTicks.

Step 3 — Stop Recalculated Every Bar (StopLossRefreshManagementEnabled, default ON)

■ *This is the mechanism that changes the stop during an active trade.*

```
// Lines 13090-13103 if (StopLossRefreshManagementEnabled) { if (!hasStopLoss &&
isAutoStopLossEnabled) { newStopLossPrice = GetInitialStopLossPrice( position.Instrument,
position.MarketPosition, multiPositionAveragePrice, // still anchored to ENTRY PRICE
quantityForCalculation); } }
```

If the recalculated price differs from the live stop order's price, the order is modified via **account.Change()**:

```
// Lines 25090-25100 foundNTOrder.StopPriceChanged = price; account.Change(new[] { foundNTOrder });
```

The critical detail is the reference point: **multiPositionAveragePrice is the entry fill price, not the current market price.** So the stop does not chase price upward. What changes every bar is the ATR itself:

Market Condition	ATR Movement	Stop Behavior
Volatility expands	ATR rises -> more ticks	Stop moves AWAY from price — gives more room
Volatility contracts	ATR falls -> fewer ticks	Stop moves TOWARD price — tightens automatically
Price moves in favor	ATR unchanged	Stop stays at same absolute price (not trailing price)
Price moves against you	ATR unchanged	Stop stays at same absolute price

Step 4 — Breakeven Trigger (Separate ATR Multiplier)

A distinct **BreakEvenAutoTriggerATRMultiplier** (line 1548) acts as a price-distance gate. Once open profit reaches ATR x BreakEvenATRMultiplier ticks, the stop is moved to breakeven automatically. This is the closest thing to trailing behavior — but it is a one-time ratchet to breakeven, not a continuous trail.

Multiplier Property	Purpose	Trailing?
StopLossInitialATRMultiplier	Stop distance — used for BOTH initial placement AND per-bar refresh	No — anchored to entry price
TakeProfitInitialATRMultiplier	Profit target distance	No
BreakEvenAutoTriggerATRMultiplier	Profit threshold that triggers breakeven move	One-time ratchet only

■ *There is NO separate trailing multiplier. The same StopLossInitialATRMultiplier applies to both the initial bracket and every subsequent per-bar update.*

Full ATR Stop Lifecycle

Stage	What Happens	Reference Point
Entry	Stop placed at entry +/- (ATR x multiplier)	Entry price
Each bar (refresh ON)	ATR recalculated; if ATR changed, account.Change() moves the stop	Still entry price
Profit threshold hit	BreakEvenAutoTriggerATRMultiplier fires; stop moved to breakeven	Entry price (breakeven)
ATM template layer	Any additional trailing (e.g. ATM built-in trail) layered on top	Managed by NT8

What This Means vs a True Trailing ATR Stop

A true trailing ATR stop follows price: for a long, the stop rises with each new high, sitting at **max(highSinceEntry) - (ATR x multiplier)**. TickHunter does not do this. Its stop stays relative to the entry price and only adapts its *width* as volatility changes. The price-following behavior is left entirely to the ATM template's built-in trailing logic.

---

4. Flatten / Exit Logic

GodZillaKilla FlattenEverything(reason) 1. AtmStrategyClose(atmStrategyId) 2. AtmStrategyClose(martingaleAtmStrategyId) 3. ExitLong / ExitShort on strategy position 4. lock(Account.Orders) — collect working orders, cancel individually	**TickHunter FlattenEverything(signalName, continueTillZero, limitInstrument)** 1. Monitor.TryEnter(FlattenEverythingLock) — reentrancy guard 2. Check 5-second throttle — prevent rapid re-fire 3. CloseAllAccountPendingOrders() — cancels pending first 4. Validate: !position.HasStateChanged() && !position.IsFlat() 5. SubmitMarketOrderChunked() — max 10 contracts/chunk with ms delay

Gaps in GodZillaKilla vs TickHunter

Gap	GodZillaKilla	TickHunter
Reentrancy protection	**None — if two triggers fire on the same bar, both execute fully**	Monitor.TryEnter() with timeout
Rapid re-fire throttle	**None — consecutive flatten calls all execute**	5-second deduplication window
Position state validation	**ExitLong/Short called without HasStateChanged() check**	!position.HasStateChanged() && !position.IsFlat() validated first
Large position handling	Single close order (fine at 1-4 contracts)	Chunked — max 10 contracts/order with ms delay between chunks

---

5. Thread Safety

Area	GodZillaKilla	TickHunter
Position iteration	lock(Account.Positions) (added v1.6.9)	Custom RealPositionService abstraction
Order iteration	lock(Account.Orders)	Custom RealOrderService abstraction
Critical path guards	None — no reentrancy protection	Monitor.TryEnter() on flatten, profit-close, pipeline
State validation	None before exit calls	!position.HasStateChanged() && !position.IsFlat()
Trade map	ConcurrentDictionary<string,TradeRecord>	N/A — no per-trade map

GodZillaKilla's collection locking is correct and sufficient for iteration safety. TickHunter goes further with full reentrancy guards on the paths that matter most. The TryGetValue single-lookup pattern is noted in TickHunter comments as a recommended practice for thread-safe dictionary access.

---

6. Summary — What GodZillaKilla Could Adopt

TickHunter Pattern	Benefit	Complexity	Priority
ATR-based initial SL/TP sizing + per-bar refresh	Bracket scales with volatility; widens in choppy conditions, tightens in calm ones. Avoids fixed stops getting blown on high-ATR days.	Medium Add ATR() buffer, CalculateATRTicks(), multiplier params	**HIGH**
Monitor.TryEnter() on FlattenEverything	Prevents double-execution when two triggers fire on the same bar	Low	**MEDIUM**

TickHunter Pattern	Benefit	Complexity	Priority
5-second throttle on FlattenEverything	Deduplicates rapid flatten calls from multiple same-direction triggers	Low	**MEDIUM**
!position.HasStateChanged() before exit	Guards against race between position flip and exit order submission	Low	**LOW**
Delayed TP/SL batching	Not needed — ATM template handles bracket natively	N/A	N/A
Chunked close orders	Not needed at typical 1-4 contract sizes	N/A	N/A

Bottom line: The most meaningful functional gap is the ATR-adaptive stop. TickHunter recalculates the stop order price every bar as volatility changes — the stop widens in choppy markets and tightens in calm ones, all anchored to entry. This is distinct from a true trailing ATR stop (which would follow price); price-following trailing is still left to the ATM template. The reentrancy lock and throttle on FlattenEverything are low-effort hardening with real protective value and should be easy wins.

Generated 2026-05-21